

République du Sénégal



MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA RECHERCHE

DIRECTION GÉNÉRALE DE L'ENSEIGNEMENT SUPÉRIEUR



Rapport DevOps — AWS CodeDeploy

Présenté par : **Ibrahima Khalilou Thiam**

Master 2 Génie Logiciel UDB

Matricule 003

À **M. Massamba LO**

Année académique 2025

Sommaire

1. [Objectifs et contexte](#)
2. [Prérequis](#)
3. [Concepts et architecture](#)
4. [Installation et configuration](#)
5. [Réalisation pratique](#)
6. [Commandes et explications](#)
7. [Vérifications et tests](#)
8. [Dépannage](#)
9. [Sécurité et bonnes pratiques](#)
10. [Checklist](#)
11. [Conclusion](#)

1. Objectifs et contexte

AWS CodeDeploy automatise le déploiement d'une révision applicative vers des instances Amazon EC2 et des serveurs on-premises. Il prend en charge des stratégies telles que le déploiement *in-place* et, selon la cible, *Blue/Green*.

- Installer et vérifier l'agent CodeDeploy.
- Préparer une révision avec `appspec.yml`.
- Créer une application et un groupe de déploiement.
- Déployer une version et suivre les événements.
- Valider l'application et diagnostiquer les échecs.

Position dans la chaîne : CodeBuild construit et teste l'artefact ; CodeDeploy prend ensuite cet artefact et orchestre son installation sur les cibles.

2. Prérequis

- Un compte AWS et une région sélectionnée.
- Une instance EC2 Linux cible avec accès réseau aux services AWS nécessaires.
- Un rôle IAM d'instance autorisant les opérations requises par l'agent.
- Un rôle de service CodeDeploy.
- AWS CLI configurée pour les opérations d'administration.
- Une application empaquetée contenant `appspec.yml`.

Architecture de laboratoire : Git/CodeCommit → CodeBuild → artefact S3 → CodeDeploy → EC2 Linux → application web.

3. Concepts et architecture

Le flux logique est : **révision** → **CodeDeploy** → **groupe de déploiement** → **agent** → **lifecycle hooks** → **validation**.

Composant	Responsabilité
Revision	Version empaquetée de l'application.
Application	Conteneur logique des déploiements.
Deployment group	Définit les instances cibles, la stratégie et les options de déploiement.
CodeDeploy agent	Reçoit la révision et exécute les opérations sur l'instance.
appspec.yml	Décrit les fichiers à installer, leurs permissions et les hooks.

4. Installation et configuration

Installation de l'agent sur Ubuntu/Debian

```
sudo apt update sudo apt install -y ruby wget cd /tmp wget https://aws-codedeploy-us-east-1.s3.amazonaws.com/latest/install chmod +x install sudo ./install auto sudo systemctl enable --now codedeploy-agent sudo systemctl status codedeploy-agent
```

Adapter l'URL S3 à la région réellement utilisée. Le statut **active (running)** indique que le service est démarré.



Préparer les rôles IAM

Le rôle de service CodeDeploy permet au service d'effectuer les opérations nécessaires au déploiement. Le rôle attaché à l'instance EC2 permet à l'agent d'accéder aux ressources nécessaires, notamment à l'emplacement de la révision.

5. Réalisation pratique

Structure de la révision

myapp/ ├── appspec.yml ├── application.jar └── scripts/ ├── stop.sh ├── start.sh └── health.sh

Exemple de appspec.yml

```
version: 0.0 os: linux files: - source: / destination: /opt/myapp overwrite: true
hooks: ApplicationStop: - location: scripts/stop.sh timeout: 60 runas: root
ApplicationStart: - location: scripts/start.sh timeout: 60 runas: root
ValidateService: - location: scripts/health.sh timeout: 60 runas: root
```

Les scripts référencés doivent être présents dans la révision et disposer des droits d'exécution nécessaires.

6. Commandes et explications

Créer l'application

`aws deploy create-application --application-name m2-webapp --compute-platform Server`
`--application-name` définit le nom logique ; `--compute-platform Server` correspond au modèle EC2/on-premises.

Créer un groupe de déploiement

```
aws deploy create-deployment-group \ --application-name m2-webapp \ --deployment-  
group-name staging \ --service-role-arn  
arn:aws:iam::123456789012:role/CodeDeployServiceRole \ --ec2-tag-filters  
Key=Environment,Value=staging,Type=KEY_AND_VALUE
```

Le filtre par tag sélectionne les instances portant `Environment=staging`. Remplacer l'ARN d'exemple par celui du rôle réellement créé.

Créer un déploiement depuis S3

```
aws deploy create-deployment \ --application-name m2-webapp \ --deployment-group-name  
staging \ --s3-location bucket=mon-bucket,key=releases/myapp.zip,bundleType=zip
```

La révision ZIP est récupérée depuis S3 puis déployée sur les instances correspondant au groupe.

7. Vérifications et tests

```
# Vérifier l'agent sudo systemctl is-active codedeploy-agent # Consulter les logs  
sudo tail -n 100 /var/log/aws/codedeploy-agent/codedeploy-agent.log # Vérifier le  
service applicatif sudo systemctl status myapp --no-pager # Tester le endpoint local  
curl --fail http://localhost:8080/health
```

Un test réussi doit confirmer à la fois le fonctionnement de l'agent, l'exécution des hooks et la disponibilité de l'application.

Validation : vérifier également dans la console CodeDeploy que le déploiement est à l'état **Succeeded** et que les événements de lifecycle sont terminés.

8. Dépannage

Symptôme	Cause possible	Action corrective
Agent arrêté	Service non démarré ou installation incorrecte	<code>systemctl status codedeploy-agent</code> puis consulter les logs.
AccessDenied	Rôle IAM incomplet	Identifier l'action refusée et appliquer le moindre privilège.
Hook failed	Script absent, non exécutable ou erreur applicative	Tester le script manuellement et vérifier les chemins.
Revision inaccessible	Accès S3/réseau insuffisant	Vérifier le rôle EC2, le bucket, la région et la connectivité.
Application indisponible	Service non démarré ou mauvais port	Vérifier <code>systemctl</code> , les logs applicatifs et le Security Group.

```
sudo journalctl -u codedeploy-agent --no-pager sudo tail -n 200  
/var/log/aws/codedeploy-agent/codedeploy-agent.log
```

9. Sécurité et bonnes pratiques

- Appliquer le principe du moindre privilège aux rôles IAM.
- Ne jamais stocker de secrets dans `appspec.yml` ou dans l'archive.
- Utiliser AWS Secrets Manager ou Systems Manager Parameter Store lorsque nécessaire.
- Limiter les accès réseau des instances avec des Security Groups restrictifs.
- Utiliser des environnements séparés pour staging et production.
- Configurer des alarmes CloudWatch et des mécanismes d'arrêt adaptés.
- Tester le rollback avant une mise en production critique.
- Conserver les logs de déploiement pour faciliter l'audit.

Attention : ne pas utiliser `AdministratorAccess` comme solution de dépannage permanente.

10. Checklist

- ☐ Instance EC2 préparée
- ☐ Agent CodeDeploy installé et actif
- ☐ Rôle IAM d'instance configuré
- ☐ Rôle de service CodeDeploy configuré
- ☐ Application CodeDeploy créée
- ☐ Groupe de déploiement créé
- ☐ `appspec.yml` validé
- ☐ Scripts de lifecycle exécutables
- ☐ Révision disponible dans S3
- ☐ Déploiement staging réussi
- ☐ Health check réussi
- ☐ Procédure de rollback vérifiée

11. Conclusion

CodeDeploy constitue la brique de déploiement automatisé de l'architecture DevOps AWS. Il permet de transformer un artefact produit par CodeBuild en une version installée et contrôlée sur les instances cibles.

L'association **CodeCommit** → **CodeBuild** → **CodeDeploy**, orchestrée par CodePipeline, permet de construire une chaîne CI/CD cohérente, reproductible et auditable. La maîtrise d'IAM, des lifecycle hooks, des logs et des mécanismes de validation est essentielle pour obtenir des déploiements fiables.

